What Is Discrete-Event Simulation?

A discrete event simulation (DES):

- tracks of what, where, and when changes happen (events)
- focus on events intead of tracking time second by second

In a discrete-event simulation, there are:

- Entities move through the system (customers, jobs, users. . .)
- Resources are limited and used by entities (servers, machines, stations,. . .)
- Processes define the sequence of steps entities follow
- Time advances from one event to the next

The Car Wash Example

We suppose that we have cars and washing stations. A car arrives at a washing station. If a washing station is free, it starts washing right away. If not, it waits in line. The car washes for some time, then finishes and leaves, making the station free for the next car.

- Entities: cars
- Resources: washing stations
- Processes: arrive $-->$ wash $-->$ depart

Events in the simulation :

Arrival $-->$ Start washing $-->$ End washing

Some definitions

- using SimJulia: load the library for discrete-event simulation
- now(sim): returns the current simulation time
- Timeout(sim, t): defines an event that occurs after t units of simulation time
- yield(event): virtually pauses a process until the event is triggered
- request(resource): request a limited resource; waits if it's busy
- unlock(resource): release the resource when done
- process: allows to create an independent process in the simulation

```
1 using Pkg
2 Pkg.add("SimJulia")
```

Discrete-Event Simulation

```
using SimJulia
using  ConcurrentSim
using ResumableFunctions

@resumable function car_washing(env::Environment
    , name::Int, stations::Resource, driving_time
    ::Number, washing_duration::Number)
@yield timeout(sim, driving_time)
println(name, " arriving at ", now(env))
@yield request(stations)
println("Car  ",name, " starting to be washed at
     ", now(env), " (waited for ", now(env) -
    driving_time, " time units)")
@yield timeout(env, washing_duration)
println(name, " leaving the stations at ", now(
    env))
@yield unlock(stations)
end
```

The simulation for the 10 Cars and 2 Washing Stations

```
#Create the simulation environment
sim = Simulation()
#Define the resource: two washing stations
stations = Resource(sim, 2)
#Define cars and arrival times
washing_duration = 5
for i in 1:10
 @process car_washing(sim, i, stations, i,
     washing_duration*2 )
 end

#Run the simulation
run(sim)
```

Results

```
1 arriving at 1.0
Car  1 starting to be washed at 1.0 (waited for 0.0 time units)
2 arriving at 2.0
Car  2 starting to be washed at 2.0 (waited for 0.0 time units)
3 arriving at 3.0
4 arriving at 4.0
5 arriving at 5.0
6 arriving at 6.0
7 arriving at 7.0
8 arriving at 8.0
9 arriving at 9.0
10 arriving at 10.0
1 leaving the stations at 11.0
Car  3 starting to be washed at 11.0 (waited for 8.0 time units)
2 leaving the stations at 12.0
Car  4 starting to be washed at 12.0 (waited for 8.0 time units)
3 leaving the stations at 21.0
Car  5 starting to be washed at 21.0 (waited for 16.0 time units)
4 leaving the stations at 22.0
Car  6 starting to be washed at 22.0 (waited for 16.0 time units)
5 leaving the stations at 31.0
Car  7 starting to be washed at 31.0 (waited for 24.0 time units)
6 leaving the stations at 32.0
Car  8 starting to be washed at 32.0 (waited for 24.0 time units)
7 leaving the stations at 41.0
Car  9 starting to be washed at 41.0 (waited for 32.0 time units)
8 leaving the stations at 42.0
Car  10 starting to be washed at 42.0 (waited for 32.0 time units)
9 leaving the stations at 51.0
10 leaving the stations at 52.0
```